

API - APPLICATION PROGRAMMING INTERFACE

Un'**API (Application Programming Interface)** è la **definizione delle interazioni consentite** tra due parti di un software.

L'API funge da contratto, specificando come un pezzo di codice o un servizio può interagire con un altro.

API - ELEMENTI DI UN CONTRATTO

L'API definisce in dettaglio il "contratto" di interazione tra il **CONSUMER** (il client) e il **PROVIDER** (il servizio). Essa specifica:

- Le **richieste** possibili.
- I **parametri** delle richieste.
- I **valori di ritorno**.
- Qualsiasi **formato di dato** richiesto (es. JSON, XML, YAML).

API - BENEFICI

L'adozione di un'API porta vantaggi fondamentali nell'architettura software:

- **Interfaccia Esplicita:** Definisce chiaramente le aspettative e le modalità di interazione.
- **Contratto Infrangibile:** Stabilisce un insieme di regole che entrambe le parti devono rispettare.
- **Information Hiding (Nascondimento delle Informazioni):** La logica interna del Provider (come i dati vengono processati o archiviati) rimane nascosta al Consumer. Il client deve solo conoscere l'interfaccia.

API - LOCALI E REMOTE

Esistono diverse categorie di API in base alla loro posizione e funzione:

- **API Locali:**
 - API per i linguaggi di programmazione (es. la libreria standard di Python).
 - API del sistema operativo.
 - API delle librerie software.
 - API hardware.
- **API Remote (Web API):**
 - Interfacce di programmazione basate su protocolli di rete (tipicamente HTTP), come le API RESTful.

API - PRIVATE VS. PUBBLICHE

La distinzione si basa sull'accessibilità:

Categoria	Descrizione	Restrizioni
API Private	Destinate all'uso interno di un'azienda o un sistema chiuso.	L'accesso è limitato ai componenti interni.
API Pubbliche	Disponibili per l'uso da parte del pubblico.	L'accesso può essere limitato solo ad alcuni utenti tramite API Tokens .

INTERFACCIA E STABILITÀ

- **Stabilità dell'Interfaccia:** I cambiamenti potrebbero rompere la compatibilità con i client esistenti.
- **Marcatori di Stato:**
 - **beta:** Indica che le parti potrebbero cambiare perché non sono ancora stabili.

- **deprecated:** Indica che le parti verranno rimosse o non saranno più supportate in futuro.

API - DOCUMENTAZIONE E DEFINIZIONE

Per stabilire il contratto, l'API deve essere definita esplicitamente.

- La definizione può avvenire attraverso **documentazione** (testo, esempi, manuali).
- Oppure, attraverso un **linguaggio di descrizione** standardizzato.

Un linguaggio di descrizione formalizza il contratto, consentendo la generazione automatica di documentazione, codice client e validazione.

OAS - OPENAPI SPECIFICATION

OAS (OpenAPI Specification) è il linguaggio di descrizione leader del settore per le API moderne basate su HTTP.

- È un formato di descrizione **vendor-neutral** (indipendente dal fornitore) per le API remote basate su HTTP.
- Rappresenta lo **standard industriale** per la descrizione delle API moderne.
- È ampiamente adottato dalla comunità di sviluppo.

API - ESEMPIO DI DOCUMENTO OPENAPI

I file OpenAPI sono tipicamente scritti in formato **YAML** (data la sua leggibilità).

Questo è un esempio di struttura base di un documento OpenAPI:

```
openapi: 3.0.0
info:
  title: An example OpenAPI document
  description: |
    This API allows writing down marks on a Tic Tac Toe board
    and requesting the state of the board or of individual cells.
  version: 0.0.1
paths: {} # Gli endpoint dell'API verrebbero definiti qui
```

Gioco Hi-Lo

- Io penso ad un numero segreto tra 1 e 100
- Tu provi ad indovinarlo con al più 10 tentativi
- ad ogni tentativo, ti dirò se il tuo tentativo era troppo alto o basso

API - REQUISITI DI SERVIZIO PER IL GIOCO HI-LO

Il Gioco Hi-Lo (indovina il numero segreto tra 1 e 100 in max 10 tentativi) richiede le seguenti funzionalità lato servizio API:

1. **Avviare** una nuova partita.
2. **Accettare** un tentativo (fino a 10) e restituire "hi" (troppo alto), "lo" (troppo basso) o "correct" (corretto).
3. **Resettare** una partita, generando un nuovo numero segreto.
4. **Ottenere** la lista di ogni tentativo in una partita, con i relativi risultati.
5. **Ottenere** la lista di tutte le partite, con il risultato finale (vittoria/sconfitta) e il numero di tentativi.

Domanda di Design: Quali **Risorse** (Resource) designeresti per soddisfare questi requisiti?



LINK

- <https://www.openapis.org>
- <https://oai.github.io/Documentation/>

